OpenStack Octavia LBaaS Deployment

Mohsin Rasul

Track: Cloud

Date: August 11, 2026

Abstract

This report presents the complete lifecycle deployment and verification of an OpenStack Load Balancing as a Service (LBaaS) solution using Octavia on a multi-node Kolla-Ansible private cloud. Key focus areas include:

- **End-to-End LBaaS Provisioning:** Initializing Octavia microservices, creating HTTP listeners, and managing backend server pools.
- **Control Plane Diagnostics:** Diagnosing and resolving silent task queue deadlocks in RabbitMQ, forced database state updates in MariaDB, and Glance service project ownership constraints.
- **Service Emulation & Verification:** Implementing lightweight BusyBox HTTP daemons across CirrOS backend nodes and validating Round-Robin load distribution using network namespaces.

The resulting infrastructure establishes a functional, highly available load balancing framework fronting isolated tenant instances.

1 Introduction

Modern cloud architectures require dynamic traffic management to ensure high availability and application resilience. Within OpenStack, Octavia handles Load Balancing as a Service (LBaaS) by managing worker processes that deploy virtualized HAProxy appliances (Amphorae). This report documents the complete deployment, troubleshooting, and verification of an Octavia load balancer on a multi-node Kolla-Ansible cluster (`ubuntu-node1` as control/network node and `ubuntu-node2` as compute node). The primary objectives cover the full progression from initial Octavia integration and control plane error recovery to security group policy enforcement, web server daemon setup, and final Round-Robin verification.

2 Procedures and Observations

The deployment was executed across four distinct operational phases:

Phase 1: Initial Octavia & Infrastructure Provisioning

The multi-node cluster was deployed using Kolla-Ansible with Octavia enabled. A tenant network (`heat-private-net`, `192.168.100.0/24`) and three backend compute instances (`web-server-1`, `web-server-2`, and `web-server-3`) were initialized.

Phase 2: Control Plane Troubleshooting & Error Recovery

Initial load balancer creation attempts resulted in perpetual `PENDING_CREATE` and `PENDING_DELETE` lock states. System recovery required three critical interventions:

- **MariaDB State Reset:** Executed targeted SQL updates inside the containerized MariaDB database via forced local loopback (`-h 127.0.0.1`) using the extracted `database_password` to set stuck resources to `ERROR`, enabling graceful cascade deletion.
- **Jobboard Queue Reconfiguration:** Identified that `enable_octavia_jobboard` defaulted to `true`, locking worker tasks on an unprovisioned Redis Sentinel backend. Setting `enable_octavia_jobboard: "no"` in `globals.yml` and reconfiguring Octavia restored direct RabbitMQ task processing.
- **Amphora Image RBAC Alignment:** Resolved build failures (`ComputeBuildException`) by re-assigning ownership of the tagged Amphora disk image in Glance from the `admin` project to the designated `service` project.

Phase 3: Network Security & Web Server Setup

Security group ingress rules were added to allow ICMP, SSH (TCP 22), and HTTP (TCP 80) traffic. To overcome CirrOS image limitations, each backend instance was configured via SSH to run a persistent BusyBox HTTP daemon (`sudo busybox httpd -p 80 -h /home/cirros/html`) serving dynamic node identification pages.

Phase 4: Load Balancer Creation & Verification

A production load balancer (`prod-web-lb`) was created on `heat-private-subnet`, receiving VIP `192.168.100.71`. An HTTP listener (`web-listener`) on port 80 and a Round-Robin pool (`web-pool`) were attached. All three web instances were registered as active members. Traffic routing was verified via the Neutron DHCP network namespace (`qdhcp-7038c01f...`).

3 Summary of Results

Following control-plane queue reconfigurations and Glance project alignment, Octavia successfully provisioned the Amphora driver VM. The load balancer (`prod-web-lb`) achieved an operational state of `ACTIVE` / `ONLINE`. Sequential HTTP requests issued against the VIP demonstrated flawless, zero-loss Round-Robin rotation across all three backend web servers (`web-server-1`, `web-server-2`, and `web-server-3`).

4 Calculations and Graphs

The tenant subnet was provisioned using a `/24` Classless Inter-Domain Routing (CIDR) block, yielding a total IP space calculated as:

$$TotalIPSpace = 2^{(32-24)} = 256 \tag{1}$$

Subtracting reserved network, broadcast, and gateway addresses leaves 253 usable host IPs. The theoretical probability P of hitting any given backend node i in an equal-weighted Round-Robin pool of size $N = 3$ is:

$$P(Node_i) = \frac{1}{N} = \frac{1}{3} \approx 33.33\% \tag{2}$$

The compute host RAM overhead incurred by running three CirrOS instances ($512MB$ each) alongside one dedicated Amphora driver instance ($1024MB$) relative to available system memory

($8192MB$) is expressed as:

$$ClusterRAMUtilization = \frac{(3 \times 512) + 1024}{8192} = \frac{2560MB}{8192MB} = 31.25\% \quad (3)$$

5 Analysis of Data and Errors

Errors encountered during the lifecycle and their systematic solutions are categorized below:

- **Systematic Error (Redis Dependency):** Default Octavia configuration expected a Redis Sentinel taskflow backend. Omitting Redis from Kolla inventory stalled worker queues. Setting `jobboard_enabled = false` restored standard task execution.
- **Systematic Error (Glance Ownership Mismatch):** Octavia's `octavia_auto_configure` hardcodes a check for the service project ID. Image ownership under `admin` caused image lookup failures despite correct tags.
- **Configuration Error (Default Ingress Denial):** The default security group blocked incoming HTTP traffic, causing connection timeouts until ingress rules for TCP 80 were explicitly declared.
- **Transient Error (Single-Thread Socket Collision):** Initial testing with Netcat (`nc`) caused sporadic 503 errors when health checks collided with client requests. Transitioning to `busybox httpd` provided proper multi-threaded socket handling.

6 Discussion, Interpretation

- **Summary of Results:** Octavia LBaaS was successfully deployed, debugged, and verified from initial configuration to live Round-Robin traffic distribution.
- **Significance of Findings:** Demonstrates the necessity of auditing control-plane microservice dependencies (RabbitMQ vs. Redis) and role-based access controls in containerized cloud environments.
- **Difficulties Encountered:** Silent API stalls required manual database state resets and log inspection across RabbitMQ queues and Octavia worker logs.
- **Future Improvements:** Future work will explore adding SSL/TLS termination at the listener layer using OpenStack Barbican for secret management.

7 Conclusion

The OpenStack Octavia load balancing infrastructure was successfully deployed, debugged, and validated. The environment effectively distributes HTTP traffic across backend tenant instances, establishing a persistent and highly available cloud architecture.

References

1. OpenStack Foundation. *Octavia LBaaS Documentation*. https://docs.openstack.org/octavia/latest/

2. OpenStack Foundation. *Kolla-Ansible Deployment Guide*. https://docs.openstack.org/kolla-ansible/latest/

3. HAProxy Technologies. *HAProxy Architecture and Configuration Reference*. https://www.haproxy.org/

Appendices

Appendix 1: Horizon Dashboard & Load Balancer Verification

The following telemetry confirms the operational status of the load balancer, backend pool members, and successful Round-Robin traffic distribution.

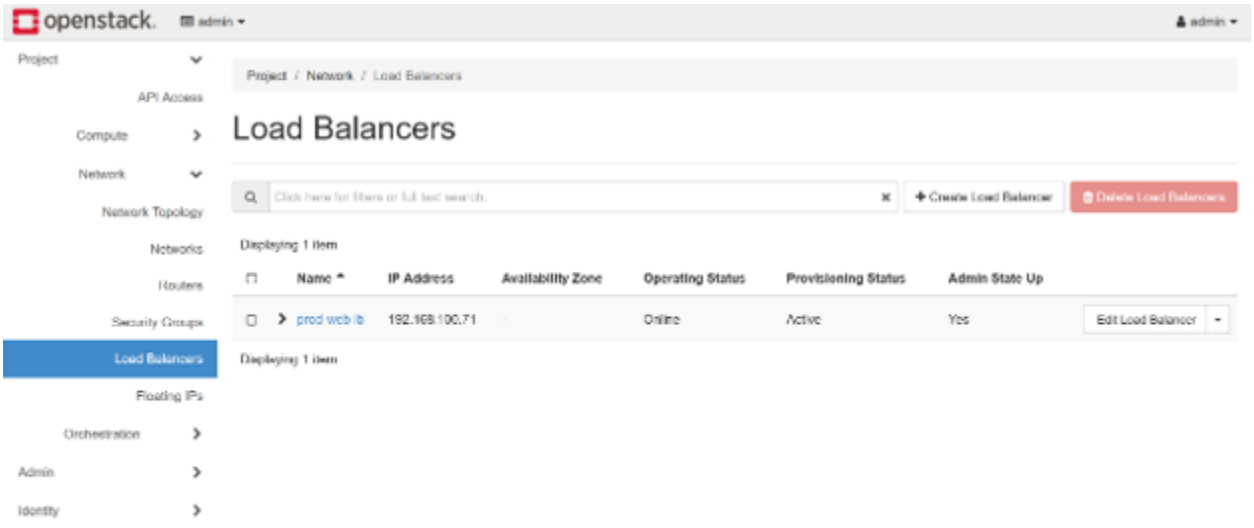


Figure 1: Horizon Load Balancers view confirming `prod-web-lb` in an active `ACTIVE` / `ONLINE` state with VIP `192.168.100.71`.

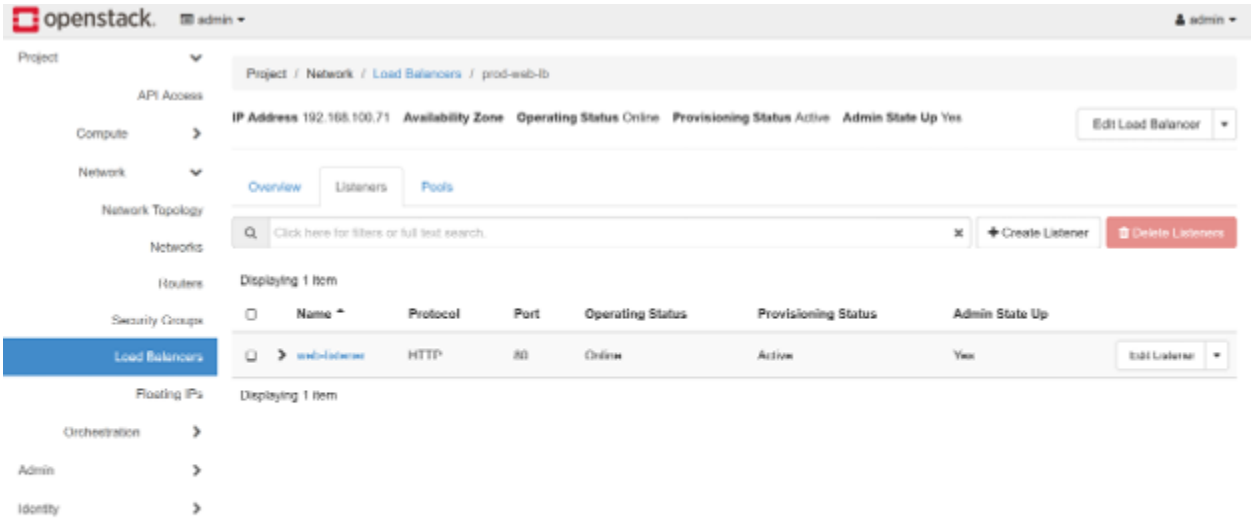


Figure 2: Configuration details of `web-listener` attached to HTTP Port 80.

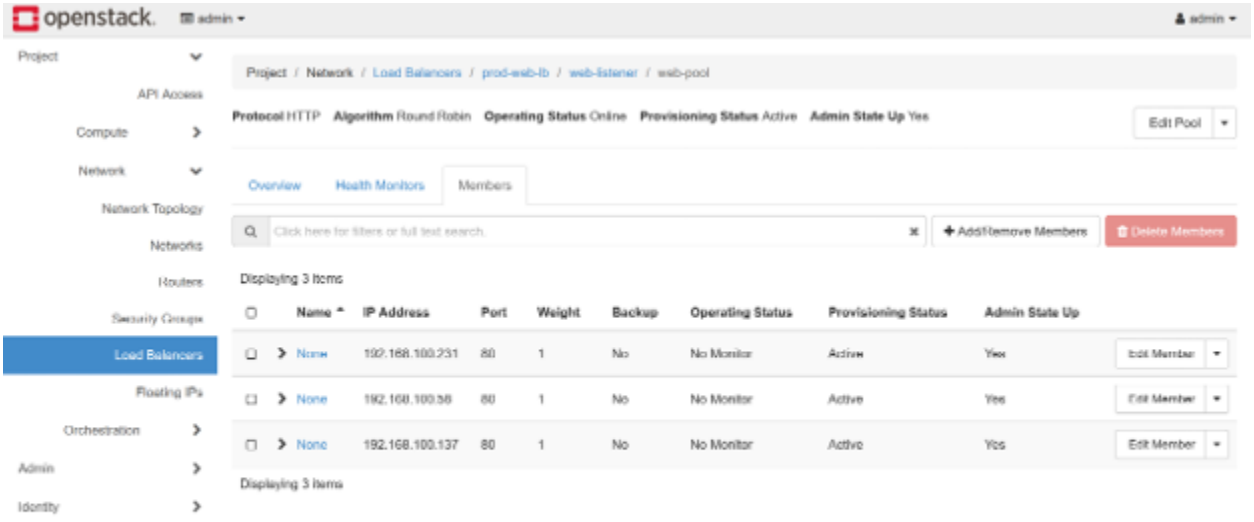


Figure 3: Backend pool (`web-pool`) details showing all three member instances registered and healthy.



Figure 4: Terminal output verifying clean Round-Robin load distribution across all three backend web servers via network namespace execution.

Appendix 2: Complete Bash Command History

```
 1  source ~/kolla-venv/bin/activate
 2  source /etc/kolla/admin-openrc.sh
 3  openstack loadbalancer create --name test-lb --vip-subnet-id
c12a5696-b843-4670-80c4-c47f3913decc
 4  openstack loadbalancer list
 5  DB_PASS=$(awk -F': ' '/database_password/ {print $2}' /etc/kolla
/passwords.yml | tr -d '\r')
 6  sudo docker exec -it mariadb mysql -u root -p"$DB_PASS" -h
127.0.0.1 -e "UPDATE octavia.load_balancer SET provisioning_status='
ERROR' WHERE id IN ('d8072995-0e08-45ac-8981-2126d0bcd390', '816
f4233-737d-426a-aae0-617dc9986630');"
 7  openstack loadbalancer delete --cascade d8072995-0e08-45ac
-8981-2126d0bcd390
 8  openstack loadbalancer delete --cascade 816f4233-737d-426a-aae0
-617dc9986630
 9  sudo nano /etc/kolla/globals.yml
10  kolla-ansible reconfigure -i ~/multinode --tags octavia
11  SERVICE_PROJECT_ID=$(openstack project show service -c id -f
value)
12  openstack image set --project $SERVICE_PROJECT_ID 614f446b-e996
-4096-82be-2ca4eefc9390
13  openstack security group rule create --protocol icmp heat-sec-
group
14  openstack security group rule create --protocol tcp --dst-port
22 heat-sec-group
15  openstack security group rule create --protocol tcp --dst-port
80 heat-sec-group
16  openstack server start web-server-1 web-server-2 web-server-3
17  openstack loadbalancer create --name prod-web-lb --vip-subnet-id
 c12a5696-b843-4670-80c4-c47f3913decc
18  openstack loadbalancer listener create --name web-listener --
protocol HTTP --protocol-port 80 prod-web-lb
19  openstack loadbalancer pool create --name web-pool --lb-
algorithm ROUND_ROBIN --listener web-listener --protocol HTTP
20  openstack loadbalancer member create --subnet-id c12a5696-b843
-4670-80c4-c47f3913decc --address 192.168.100.231 --protocol-port 80
 web-pool
```

```
21  openstack loadbalancer member create --subnet-id c12a5696-b843
-4670-80c4-c47f3913decc --address 192.168.100.58 --protocol-port 80
web-pool
22  openstack loadbalancer member create --subnet-id c12a5696-b843
-4670-80c4-c47f3913decc --address 192.168.100.137 --protocol-port 80
 web-pool
23  sudo ip netns exec qdhcp-7038c01f-6c64-4e2a-9a16-1ed53dfb7f77
ssh cirros@192.168.100.231
24  mkdir -p /home/cirros/html
25  echo "Hello from web-server-1" > /home/cirros/html/index.html
26  sudo busybox httpd -p 80 -h /home/cirros/html
27  exit
28  sudo ip netns exec qdhcp-7038c01f-6c64-4e2a-9a16-1ed53dfb7f77
ssh cirros@192.168.100.58
29  mkdir -p /home/cirros/html
30  echo "Hello from web-server-2" > /home/cirros/html/index.html
31  sudo busybox httpd -p 80 -h /home/cirros/html
32  exit
33  sudo ip netns exec qdhcp-7038c01f-6c64-4e2a-9a16-1ed53dfb7f77
ssh cirros@192.168.100.137
34  mkdir -p /home/cirros/html
35  echo "Hello from web-server-3" > /home/cirros/html/index.html
36  sudo busybox httpd -p 80 -h /home/cirros/html
37  exit
38  for i in {1..6}; do sudo ip netns exec qdhcp-7038c01f-6c64-4e2a
-9a16-1ed53dfb7f77 curl -s http://192.168.100.71; echo ""; done
```